

通用识别使用开发文档 —— generalOCRv1.0

简介

此软件可以对增值税发票，专票，身份证正反面，各种类型车牌，表格图片，文档pdf，银行卡进行识别并对其中关键信息进行对应提取总结。此软件可以在完全离线的环境下使用本地服务器的gpu或者cpu对图像进行识别提取。

使用说明

在服务器主机通过dockerfile创建docker部署虚拟环境

运行服务器docker镜像：使用命令 `docker run -it -p 5000:5000 <new_image_name>`

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
invoiceocr	latest	70db86c2238b	41 minutes ago	6.07GB

(此处IMAGE ID可能有变化建议使用image_name)

有两种使用方式：

- 1.调用API接口输入单张发票照片路径还有需要识别的项目名称
- 2.在一个文件夹里放置任意张待识别发票照片，输出结果自动生成在一张csv表格里包括所有项目类别的结果

API接口调用说明（使用方法1）

详见generalOCR接口使用文档

批量图片处理和表格自动化生成说明（使用方法2）

在docker里的根路径下可以看到images这个文件夹，将要识别的发票图片(jpg, png,pdf)格式全部放进去

使用命令 `python general_ocr.py` 启动程序

1. 发票识别
2. 身份证识别
3. 车牌识别
4. 表格识别
5. 文档识别
6. 银行卡识别

选择一个要识别的文件类型

它会识别所有images文件夹里的图片pdf并将最终生成的识别结果保存在"识别结果.csv"里 (对于1,2,3,6), (对于4, 5)会保存在单独的文件里。它们都会保存在images文件夹中

代码说明

general_ocr_api.py:

因为general_ocr_api.py和general_ocr.py的功能相近所以此处直接使用general_ocr_api.py的代码进行详解

库 cv2: OpenCV库, 用于图像处理。 numpy: 数值计算库。 re: 正则表达式库, 用于字符串处理。 PaddleOCR、PPStructure、save_structure_res: PaddleOCR库, 用于OCR处理及结果保存。 pandas: 数据分析库。 BeautifulSoup: 用于解析HTML和XML。 io: Python内置的输入输出操作库。 xlsxwriter: 用于创建Excel文件。 sorted_layout_boxes, convert_info_docx: 从PaddleOCR中导入的函数, 用于文档恢复。 os: 操作系统接口库。 Document: 用于创建和操作Word文档。 shutil: 高级文件操作库。 sys: 与Python解释器和环境相关的功能。 contextmanager: 上下文管理器, 用于简化资源管理。

图像预处理:

1.图像摆正

首先用open cv去检测图片是否是摆正的, 如果没摆正就调用rotate_image对它旋转一定角度。

经过实验我发现所有图片旋转2次可以让所有角度拍摄的图片都回正所以我写了correct_image_orientation_twice这个函数, 然后我根据图片的每次识别平均置信度的读取来判断是否需要继续旋转图片还有需要再旋转多少角度

```
rotate_image(image, angle) #根据指定的角度旋转图像
```

```
correct_image_orientation(image) #将图像转换为灰度图像, 然后使用Canny边缘检测和霍夫变换检测直线, 从而校正图像方向。
```

```
correct_image_orientation_twice(image_path) #读取图像并调用两次correct_image_orientation 函数, 以提高校正精度。
```

Sample invoice content redacted

```
confidence = ocr_confidence(result)
if confidence < 0.96:
    # Rotate the image 180 degrees and re-run OCR
    corrected_image = rotate_image(corrected_image, 180)
    result = ocr.ocr(corrected_image, cls=True)
```

(optional) : 你可以在 corrected_image 这个文件夹里查看预处理过的发票照片

2. 对图像进行锐度化灰度化和对比度的处理

```
# image pre processing
def process_image(image):
    # Convert the image to a grayscale image
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Create a sharpening filter
    kernel = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])

    # apply the sharpening filter onto the image
    sharpened_image = cv2.filter2D(gray_image, -1, kernel)

    # create a CLAHE instance
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))

    #apply CLAHE to increase the image's contrast
    contrast_image = clahe.apply(sharpened_image)

    # adjust contrast and brightness
    alpha = 1.5 # contrast control (1.0-3.0)
    beta = -50 # brightness control (-100 to 100)

    adjusted_image = cv2.convertScaleAbs(contrast_image, alpha=alpha, beta=beta)
    return adjusted_image
```


通过灰度化简化图像数据，仅保留亮度信息，从而专注于文本的亮度变化，提高OCR识别的准确性。

使用卷积核对图像进行滤波，增强图像的边缘细节。

使用CLAHE直方图均衡化方法，通过在小区域内均衡化图像直方图来增强局部对比度，避免过度增强噪声。可以改善光照不均的图像质量，从而提高OCR的识别率。

通过线性变换调整图像的亮度和对比度，alpha控制对比度，beta控制亮度。

图像检测:

使用开源的PaddleOCR基于深度学习的模型和自己预训练的模型进行图像检测

对于文档检测，通过假设待检测文档为全英文进行识别如果识别结果置信度过低使用适用其他多语言的模型再次进行识别来提升检测精度

识别结果保存在列表txts里

```
txts = [line[1][0] for line in result[0]]
```

或者存储在了text这个文本里

```
for i in range(len(txts)):
    txts[i] = txts[i].replace(" ", "")
text = ' '.join(txts)
```

内容提取:

在图像识别后生成的结果是非常混乱的并且可能会包含外部干扰文字, 使用正则表达式的方法提取内容，通过检测其关键字的出现，关键字前后关系，关键字前后文字类型，文本段的长度结构判断其语义从而提取出不同识别项目对应结果，最后将其存放在字典里返回。

Sample invoice content redacted

(及未经处理OCR识别结果的且按制证头例)

具体各个文件类型提取信息的实现：

fp_ocr(发票识别):

```
corrected_image, angle = correct_image_orientation_twice(image)#对图像进行两次旋转预
处理发票变成0度或180度
corrected_image = process_image(corrected_image)
result = ocr.ocr(corrected_image, cls=True)#进行ocr识别
confidence = ocr_confidence(result)#获得平均置信度
if confidence < 0.96:#如果置信度低说明发票角度为180度进行一个180度旋转回正
    corrected_image = rotate_image(corrected_image, 180)
    result = ocr.ocr(corrected_image, cls=True)
```

#信息提取

#使用正则表达式

```
dic['发票代码'] = re.findall(r'\d{12}', text)[0] if re.findall(r'\d{12}', text) else ''
dic['发票号码'] = re.findall(r'\d{8}', text)[0] if re.findall(r'\d{8}', text) else ''
dic['开票日期'] = re.findall(r'[\d ]+年[\d ]+月[\d ]+日', text)[0] if
re.findall(r'[\d ]+年[\d ]+月[\d ]+日', text) else ''
dic['校验码'] = re.findall(r'[\d ]{20,30}', text)[0] if re.findall(r'[\d ]{20,30}',
text) else ''
dic['税率'] = re.findall(r'免税|不征税|1{0,1}[1369]%', text)[-1] if re.findall(r'免
税|不征税|1{0,1}[1369]%', text) else ''
dic['价税合计(小写)'] = re.findall(r'[¥|¥]{0,1}\d+\.\d+', text)[-1] if
re.findall(r'[¥|¥]{0,1}\d+\.\d+', text) else ''
dic['购买方名称'] = gongsi[0] if gongsi else ''
dic['销售方名称'] = gongsi[-1] if gongsi else ''

tax_ids = re.findall(r'\b[0-9A-Z]{15,18}\b', text)
dic['购买方纳税人识别号'] = f'"{tax_ids[0]}"' if len(tax_ids) > 0 else ''
dic['销售方纳税人识别号'] = f'"{tax_ids[1]}"' if len(tax_ids) > 1 else ''

bank_accounts = re.findall(r'[-一-龟]+银行[-一-龟\d]*[\dA-Z]*', text)
```

```

shou_index = text.find('售')
xiao_index = text.find('销')
dic['购买方开户行及账号'] = ''
dic['销售方开户行及账号'] = ''
for account in bank_accounts:
    account_index = text.find(account)
    #检测销和售这两个字的位置来辨别发票上同样title的“开户行及账号”，“纳税人识别号”，“名称”哪个是销售方的哪个是购买方的
    if (shou_index != -1 and account_index > shou_index) or (xiao_index != -1 and account_index > xiao_index):
        dic['销售方开户行及账号'] = account
    else:
        dic['购买方开户行及账号'] = account

dic['开票日期'] = re.sub(r'年|月|日| ', '', dic['开票日期'])
dic['校验码'] = dic['校验码'].replace(' ', '')

return dic

```

cp_ocr(车牌识别)

```

#通过平均置信度对车牌进行180度翻转回正
corrected_image, angle = image, 0
result = ocr.ocr(corrected_image, cls=True)
confidence = ocr_confidence(result)
if confidence < 0.96:
    corrected_image = rotate_image(corrected_image, 180)
    result = ocr.ocr(corrected_image, cls=True)

```

```

provinces = ["皖", "沪", "津", "渝", "冀", "晋", "蒙", "辽", "吉", "黑", "苏", "浙", "京", "闽", "赣", "鲁", "豫", "鄂", "湘", "粤", "桂", "琼", "川", "贵", "云", "藏", "陕", "甘", "青", "宁", "新", "警", "学"]
specials = ["WJ", "军", "警", "使", "领"]
alphabets = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'J', 'K', 'L', 'M', 'N', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'O']
for i in range(len(txts)):
    item = txts[i]
    #通过检测省份名特殊字符，字母是否存在在车牌特定位置判断txts这个识别文本列表中哪一段或者说哪几段合在一块是合法车牌从而排除干扰信息
    if any(province in item for province in provinces) and any(alphabet in item for alphabet in alphabets) or any(special in item for special in specials) and any(alphabet in item for alphabet in alphabets):
        if len(item) < 4:
            if i+1 in range(len(txts)):
                item = item + txts[i+1]
            else:
                item = item + txts[i-1]
        print(item)
        dic['车牌号'] = item
        break

```



```
return dic
```

id_ocr(身份证识别)

```
corrected_image, angle = image, 0
#对图像进行灰度化锐化滤波对比度处理
corrected_image = process_image(corrected_image)
result = ocr.ocr(corrected_image, cls=True)
txts = [line[1][0] for line in result[0]]
for i in range(len(txts)):
    txts[i] = txts[i].replace(" ", "")
```

#判断输入照片是身份证正面还是反面从而提取不同信息

```
if "中华人民共和国" in txts:
    is_front_cover = True
```

#提取正面信息

```
if is_front_cover:
    for item in txts:
        if item != "居民身份证" and item != "中华人民共和国" and item != "签发机关"
and item != "有效期限":
            if is_date(item):
                dic['有效期限'] = item
            elif "有效期限" in item:
                dic['有效期限'] = item[4:]
            elif "签发机关" in item:
                dic['签发机关'] = item[4:]
            else:
                dic['签发机关'] = item
```

#当输入图片为身份证反面时

```
corrected_image = image
```

```
i = 0
```

#对身份证进行4次旋转当姓名这两个字出现在第一位时说明身份证照片是正的，因为ocr是从左往右从上往下识别的

```
while "姓名" not in txts[0] and i < 5:
    corrected_image = rotate_image(corrected_image, 90)
    result = ocr.ocr(corrected_image, cls=True)
    txts = [line[1][0] for line in result[0]]
    i += 1
    print(txts)
```

#当旋转四次之后依然没有出现姓名说明身份证读取存在问题

```
if i == 5:
    print('识别失败: {}'.format("name"))
```

```

keys = ["姓名", "性别", "民族", "出生", "住址", "公民身份号码"]

# extract useful information from the text
for i, key in enumerate(keys):
    start_index = text.find(key) + len(key)
    if i < len(keys) - 1:
        end_index = text.find(keys[i + 1])
    else:
        end_index = len(text)
    dic[key] = str(text[start_index:end_index].strip())

#Further process the date of birth
birth_date = dic['出生']
birth_date_parts = re.findall(r'\d+', birth_date)
if len(birth_date_parts) == 3:
    dic['出生年月日'] = f"{birth_date_parts[0]}年{birth_date_parts[1]}月{birth_date_parts[2]}日"
else:
    dic['出生年月日'] = birth_date

#Remove the original birth field
del dic['出生']

return dic
#用于身份证正面有效期限的提取，判断格式是不是合法
def is_date(date_string):
    pattern = r'^\d{4}\.\d{2}\.\d{2}-\d{4}\.\d{2}\.\d{2}c3637169-7a05-425d-92ab-35e5a2a01706-1718613337783#39;
    if re.match(pattern, date_string):
        return True
    else:
        return False

```

bc ocr(银行卡识别)

银行卡使用的是一个自定义模型需要转移到另一个工作路径和环境路径

```

#改变工作路径
@contextmanager
def change_working_directory(new_dir):
    original_dir = os.getcwd()
    try:
        os.chdir(new_dir)
        yield
    finally:
        os.chdir(original_dir)

```



```

#改变环境路径
original_paths = sys.path
paths = [
    "/app/data",
    "/app/data",
    "/app/data",
    "/app/data",
    "/app/data",
    "/app/data/deploy",
    "/usr/local/lib/python37.zip",
    "/usr/local/lib/python3.7",
    "/usr/local/lib/python3.7/lib-dynload",
    "/usr/local/lib/python3.7/site-packages",
    "/app/data/tools/infer",
    "/app/data/tools/infer",
    "/app/data/tools/infer",
    "/app/data/tools/infer"
]
sys.path = paths

```

因为银行卡只要提取银行卡号直接读取即可，银行卡类型和银行名称提取目前版本需要联网

excel ocr(excel识别恢复)

```

#初始化ppstructure对excel表格进行识别
table_engine = PPStructure(recovery=True, lang='ch', use_gpu = gpu)

```

```

for item in result:
    #判断识别结果是否为表格
    if item['type'] == 'table':
        #提取表格内容
        html_content = item['res']['html']
        soup = BeautifulSoup(html_content, 'html.parser')
        table = soup.find('table')
        df = pd.read_html(str(table))[0]

        以流的形式将恢复的excel表格返回给api服务程序
        output = io.BytesIO()
        writer = pd.ExcelWriter(output, engine='xlsxwriter')
        df.to_excel(writer, index=False)
        writer.save()
        output.seek(0)
        return output

```

doc ocr(文档识别恢复)

doc_ocr_helper通过将doc pdf图片流根据页码将每页图片存储在一个零时文件夹并对每张图片进行预处理，调用doc_ocr会存储文字内容位置表格图片信息，并输出对于当前页码图片的docx恢复文

档

当所有图片检测完时会调用merge_docx_files对于每页图片恢复的docx文件进行合并最后合并成一个完整的docx文件

```
def merge_docx_files(source_file1, source_file2, output_file):
    #open the first document
    doc1 = Document(source_file1)

    #open the second document
    doc2 = Document(source_file2)

    # Add the content of the second document to the end of the first document.
    for element in doc2.element.body:
        doc1.element.body.append(element)

    # save the merged document
    doc1.save(output_file)
```

删除零时文件夹里的所有文件

```
if os.path.exists(folder_paths[i]) and os.path.isdir(folder_paths[i]):
    try:
        # recursively delete all the non-empty folders
        shutil.rmtree(folder_paths[i])
        print(f"非空文件夹 '{folder_paths[i]}' 及其内容已成功删除。")
    except Exception as e:
        print(f"删除文件夹时出错: {e}")
```

最后以二进制流的形式返回最终的docx文件给api服务程序

general_API_service.py:

库:

Flask: 用于构建 Web 应用程序，用来提供API接口服务

numpy: 提供了对多维数组对象的支持，以及用于操作这些数组的各种函数。

cv2: OpenCV 库

urllib.parse: urllib.parse 提供了解析和操作 URL 的方法。

json: 进行 JSON 数据的编码和解码。

general_ocr_API: 对于不同种类文件类型的信息检测提取程序

pdf2image:可以将 PDF 文件转换为图像（如 JPEG、PNG）的库，便于处理 PDF 文件中的页面。

io: 提供了 Python 的核心 I/O 功能，用于处理流和文件系统等。

参数字典：

翻译字典让输入参数为中文或者英文都可以被程序识别和理解，同时也可以用来判断参数是否合法

```
trans = {
    "bank_card": "银行卡",
    "bank_card_number": "银行卡号",
    "bank_name": "银行名称",
    "card_type": "卡类型",
    "invoice": "发票",
    "license_plate": "车牌",
    "id_card": "身份证",
    "excel": "表格",
    "file_name": "文件名",
    "license_no": "车牌号",
    "name": "姓名",
    "invoice_code": "发票代码",
    "gender": "性别",
    "invoice_no": "发票号码",
    "ethnicity": "民族",
    "invoice_date": "开票日期",
    "birthdate": "出生年月日",
    "verification_code": "效验码",
    "address": "住址",
    "tax_rate": "税率",
    "id_number": "公民身份号码",
    "total_amount_with_tax": "价税合计(小写)",
    "issuing_authority": "签发机关",
    "buyer_taxpayer_identification_no": "购买方纳税人识别号",
    "validity_period": "有效期限",
    "seller_taxpayer_identification_no": "销售方纳税人识别号",
    "buyer_name": "购买方名称",
    "seller_name": "销售方名称",
    "buyer_bank_and_account_no": "购买方开户行及账户",
    "seller_bank_and_account_no": "销售方开户行及账户",
}
```

#程序从接口收到的三个参数

```
category = request.args.get('category', 'text')
```

```
doc_type = request.args.get('type')
```

```
use_gpu = request.args.get('use_gpu', 'false').lower() == 'true'
```

#对中文字符进行解码

```
category = category.encode('latin1').decode('utf-8')
```

```
doc_type = doc_type.encode('latin1').decode('utf-8')
```



```

#将pdf图片每页进行转换和存储
if file.filename.lower().endswith('.pdf'):
    # Convert PDF to image
    pdf_bytes = file.read()
    images = convert_from_bytes(pdf_bytes)
    if images:
        image = np.array(images[0])
        image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR) # Convert from PIL to
OpenCV format
    else:
        return jsonify({'error': 'Error converting PDF to image'}), 400

```

```

#对于接口返回类型是文件的操作
if doc_type == 'excel':
    excel_stream = excel_ocr(image)
    if excel_stream:
        return send_file(excel_stream, download_name="result_table.xlsx",
as_attachment=True)
    else:
        return jsonify({"error": "Failed to process image"}), 500
print("just before identifying doc type")

# return file-based ocr result in dictionary form
if doc_type == 'document':
    print("recognized doc_type")
    if file.filename.lower().endswith('.pdf'):
        doc_stream = doc_ocr_helper(images)
    else:
        images = []
        images.append(image)
        doc_stream = doc_ocr_helper(images)
    print(doc_stream)
    if doc_stream:
        print('file sent')
        return send_file(doc_stream, download_name="result_doc.docx",
as_attachment=True)
    else:
        return jsonify({"error": "Failed to process image"}), 500

```

```

#对于接口返回类型是json文本的, 调用general_ocr_API.py并返回一个字典
if doc_type == 'id_card':
    data = id_ocr(image)
elif doc_type == 'license_plate':
    data = cp_ocr(image)
elif doc_type == 'invoice':
    data = fp_ocr(image)
elif doc_type == 'bank_card':
    data = bc_ocr(image)
else:

```

```
return {'error': 'Invalid document type'}
```

```
#返回response
```

```
response = make_response(json.dumps(result, ensure_ascii=False))
```

```
response.headers['Content-Type'] = 'application/json; charset=utf-8'
```

```
return response
```